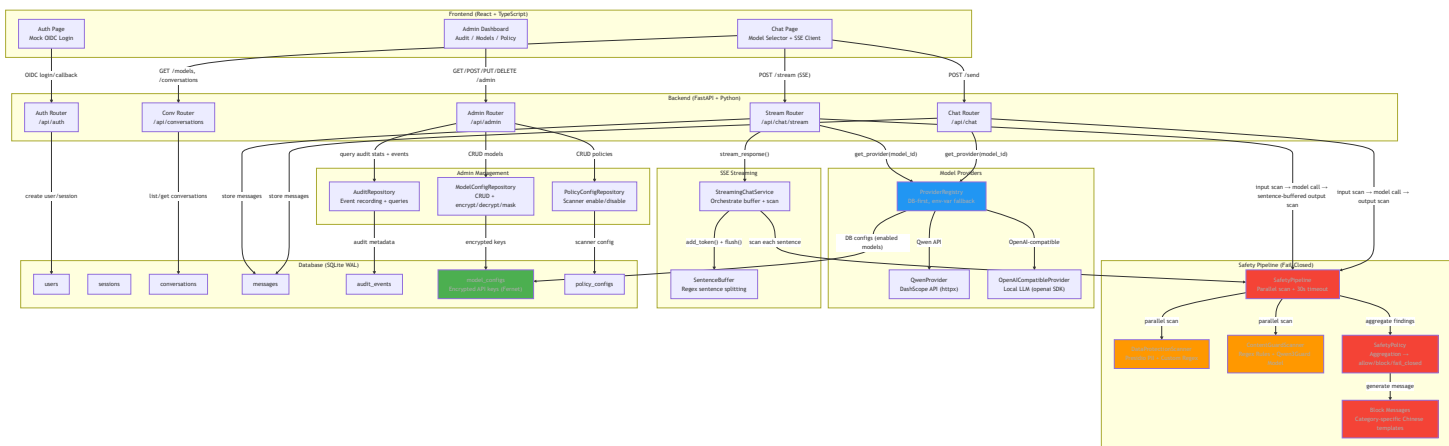


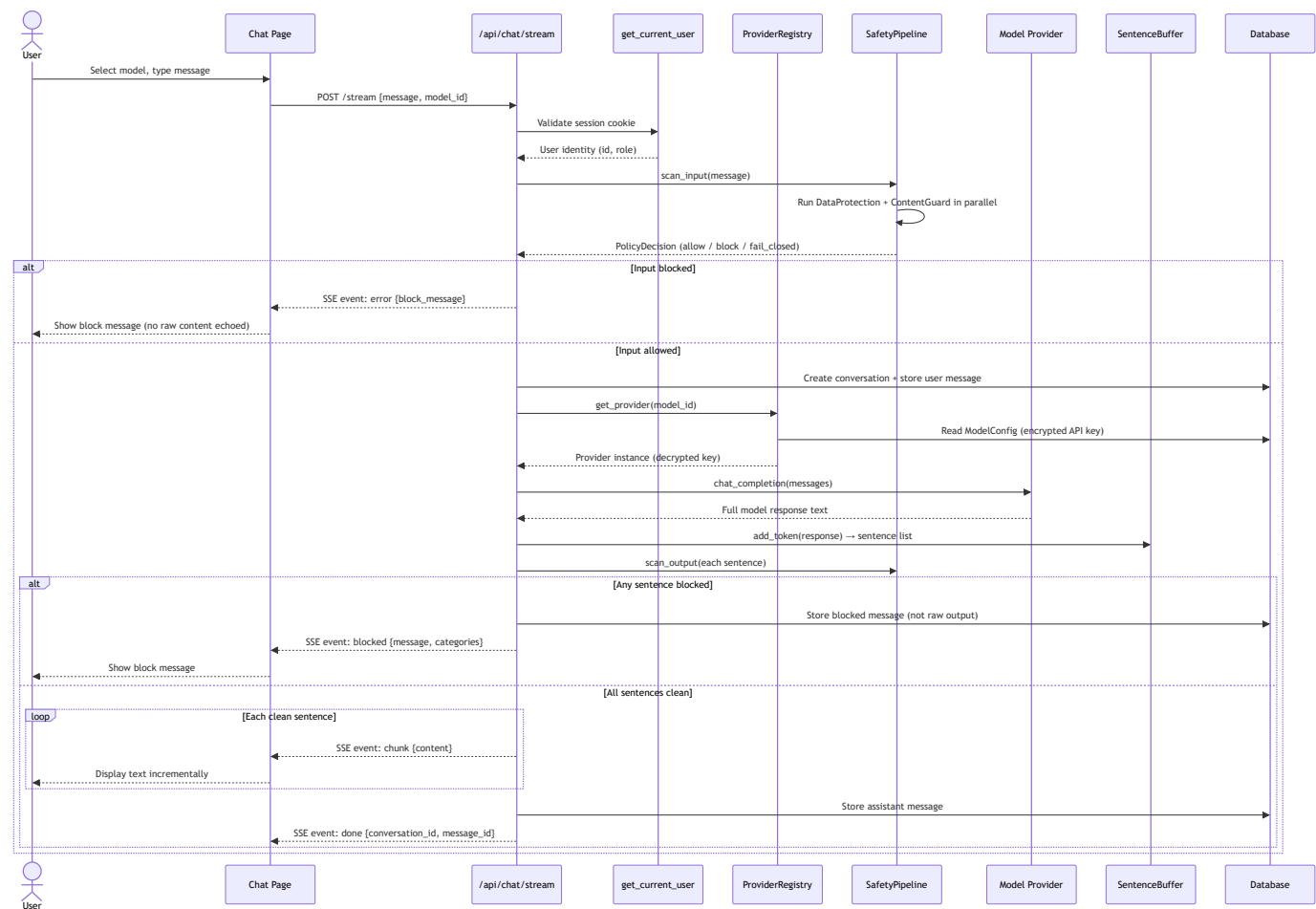
Enterprise AI Chat MVP — System Architecture

Architecture Overview

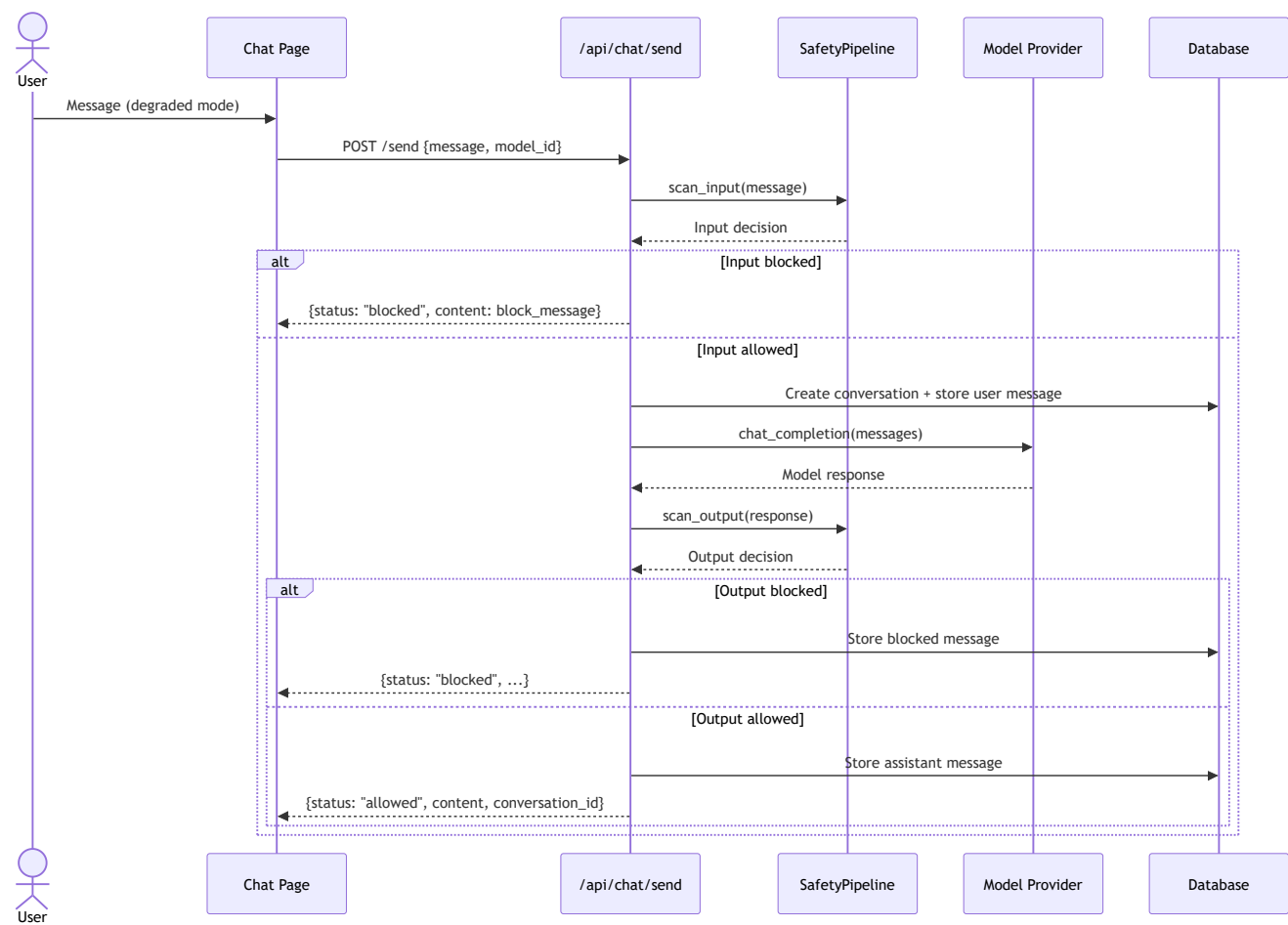
Enterprise AI Chat MVP uses a safety-gated request pipeline architecture: every user message and model response passes through a multi-scanner safety pipeline before proceeding. The system enforces fail-closed semantics — if any scanner crashes, times out, or returns ambiguous results, the content is blocked rather than allowed unfiltered.



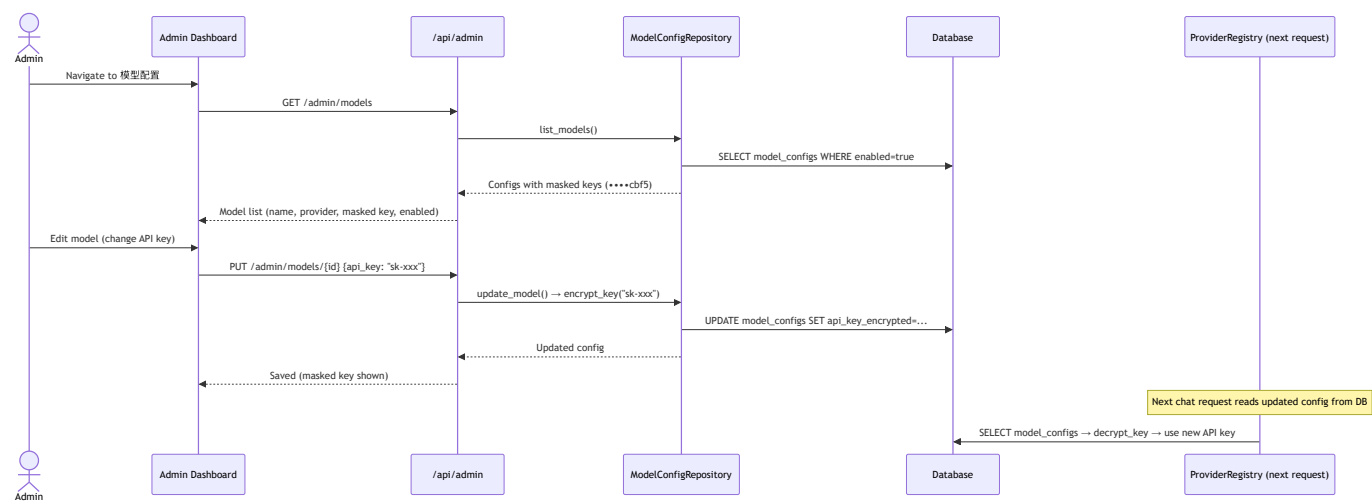
Request Flow — Streaming Chat (Primary Path)



Request Flow — Non-Streaming Chat (Fallback)



Admin Configuration Flow



Component Details

Component	File	Purpose
SafetyPipeline	backend/app/safety/pipeline.py	Coordinates all scanners in parallel with 30s timeout; fail-closed on crash/timeout
DataProtectionScanner	backend/app/safety/data_protection.py	Presidio PII detection (regex-only entities: EMAIL, PHONE, SSN, CREDIT_CARD, etc.) + custom recognizers (Chinese national ID, employee ID, income, project code). Score threshold 0.7 to avoid Chinese false positives
ContentGuardScanner	backend/app/safety/llm_guardrails.py	Three-layer detection: (1) regex rules for prompt injection/jailbreak patterns (Chinese + English), (2) regex rules for harmful content requests (hate speech, discrimination), (3) Qwen3Guard-Gen-0.6B generative model for probabilistic detection of content rules don't cover
SafetyPolicy	backend/app/safety/policy.py	Aggregates all scanner findings → single allow/block/fail_closed decision. Any violation = full block, no partial display
Block Messages	backend/app/safety/block_messages.py	Chinese-language category-specific templates. Never echoes detected content — only category labels and revision hints

Component	File	Purpose
ProviderRegistry	backend/app/models/providers.py	DB-first: reads ModelConfig table with encrypted API keys. Falls back to env vars only if DB empty. Admin changes take effect on next request
ModelConfigRepository	backend/app/admin/models_repo.py	CRUD + Fernet encryption/decryption/masking of API keys. mask_key() shows last 4 chars only
StreamingChatService	backend/app/streaming/service.py	Orchestrates: input scan → model call → SentenceBuffer splits response → per-sentence output scan → SSE events. Commits at each write boundary to avoid holding SQLite write lock
SentenceBuffer	backend/app/streaming/buffer.py	Regex sentence boundary detection. Accumulates tokens, flushes complete sentences